

# Sobreposição de computação e comunicação

Eugenio Vitor Lopes dos Santos

DCA3703, Programação Paralela, Universidade Federal do Rio Grande do Norte

## 1 Enunciado

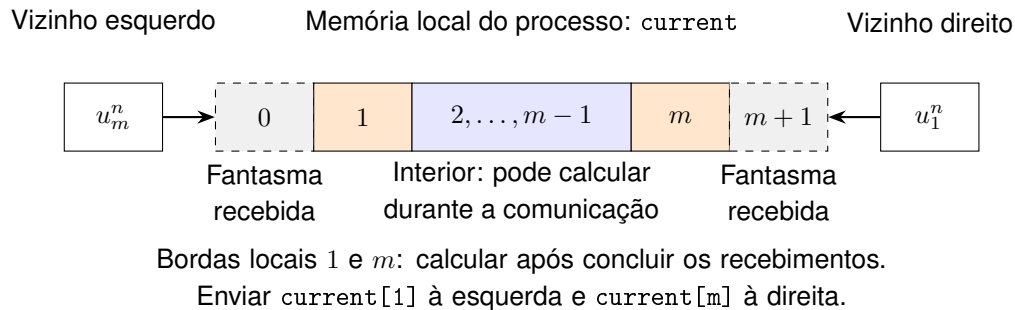
Simular a difusão de calor em uma barra 1D distribuída entre dois ou mais processos MPI, com células extras para trocar as bordas. Comparar três versões: comunicação com `MPI_Send` e `MPI_Recv`; comunicação não bloqueante com `MPI_Isend`, `MPI_Irecv` e `MPI_Wait`; e uma versão que usa `MPI_Test` enquanto calcula os pontos internos.

## 2 Fundamentação

Cada processo armazena um trecho da barra e duas células fantasma, uma de cada lado. A atualização de uma célula usa seu valor e os valores das duas vizinhas no passo anterior:

$$u_i^{n+1} = u_i^n + 0,2 (u_{i-1}^n - 2u_i^n + u_{i+1}^n).$$

As células das extremidades locais dependem dos valores recebidos dos processos vizinhos. As células estritamente internas não dependem dessa troca e podem ser atualizadas enquanto ela está pendente. Nas extremidades globais da barra, as células fantasma permanecem em zero.



**Figura 1:** Índices locais, com  $m = \text{count}$ . As setas mostram os recebimentos de um processo com dois vizinhos; cada mensagem contém um `double`. Nas extremidades globais, o vizinho ausente é `MPI_PROC_NULL` e a célula fantasma permanece em zero.

Na Figura 1, atualizar  $i = 1$  exige ler `current[0]`, enquanto atualizar  $i = m$  exige ler `current[m+1]`. Para  $2 \leq i \leq m-1$ , os três valores necessários já pertencem ao processo. Todas as atualizações leem `current` e escrevem em `next`; assim, nenhum valor do passo  $n$  é substituído por um valor do passo  $n+1$  durante o cálculo.

## 2.1 Troca bloqueante

`MPI_Send` e `MPI_Recv` trocam as bordas antes de qualquer atualização. O código envia primeiro à direita e recebe da esquerda; em seguida, envia à esquerda e recebe da direita. `MPI_PROC_NULL` representa o vizinho ausente nas extremidades globais.

## 2.2 Comunicação ponto a ponto não bloqueante

As operações não bloqueantes retornam sem exigir que a transferência esteja concluída. Enquanto um recebimento estiver pendente, as células fantasma correspondentes não podem ser usadas no cálculo.

### 2.2.1 Escondendo latência com sobreposição de computação e comunicação

Após iniciar a troca, o processo pode atualizar as células internas, que não dependem das bordas recebidas. Só depois da conclusão dos recebimentos atualiza as duas extremidades locais. A versão com `MPI_Wait` espera antes de calcular; a versão com `MPI_Test` consulta os recebimentos durante o cálculo interno. Essa consulta também tem custo.

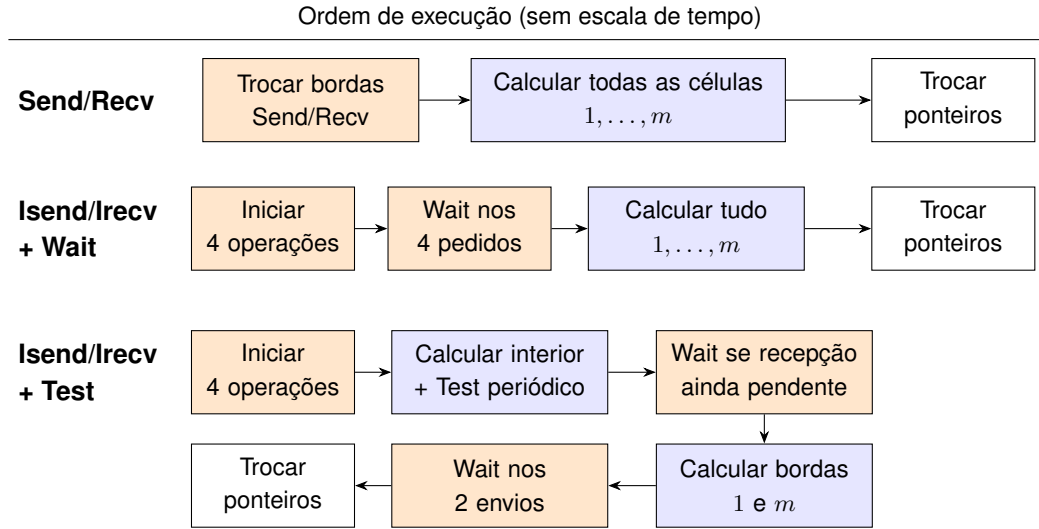
### 2.2.2 Criação de `MPI_Request`

- `MPI_Isend`: inicia o envio de uma borda e devolve um `MPI_Request` para acompanhar a operação.
- `MPI_Irecv`: inicia o recebimento de uma borda e devolve um `MPI_Request` para acompanhar a operação.

### 2.2.3 Consumo de `MPI_Request`

- `MPI_Wait`: espera a conclusão da operação associada ao `MPI_Request`.
- `MPI_Test`: verifica se a operação terminou sem bloquear; se ainda estiver pendente, o pedido continua válido.

## 2.3 Ordem das operações em cada passo



**Figura 2:** Sequência implementada pelas três versões. Na terceira, seguir as setas até a linha inferior. O cálculo interno oferece uma oportunidade de sobreposição; o diagrama não representa durações medidas nem garante progresso simultâneo das transferências.

Na versão com `MPI_Wait`, os quatro pedidos são concluídos antes da primeira atualização. Na versão com `MPI_Test`, o laço calcula  $i = 2, \dots, m - 1$  e consulta os recebimentos ainda pendentes quando  $i \% 1024 == 0$ . O teste informa a conclusão por uma flag; se ela continuar falsa ao terminar o interior, `MPI_Wait` garante que a célula fantasma esteja pronta antes do cálculo das bordas.

Os pedidos `requests[0]` e `requests[1]` acompanham os recebimentos; os pedidos `requests[2]` e `requests[3]`, os envios. Enquanto um envio estiver pendente, seu buffer em `current` não pode ser alterado. Escrever os resultados em `next` permite calcular sem modificar esse buffer. A espera pelos envios ao final libera sua reutilização nos passos seguintes; só então o programa troca os ponteiros dos dois vetores.

## 3 Experimento

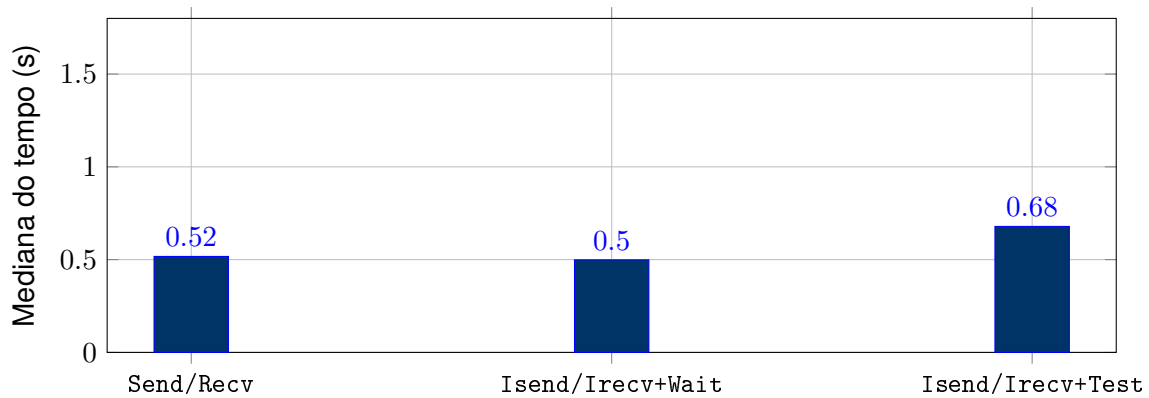
O job 2109344 foi executado na partição `intel-512` do NPAD, com dois processos em nós distintos, uma CPU por processo. O programa foi compilado com `mpicc -std=c11 -O3 -Wall -Wextra`. A barra tinha 1 048 576 células, distribuídas igualmente; cada processo possuía 524 288 células locais e duas fantasmas. A metade esquerda começou com temperatura 1, e a metade direita com 0. Foram executados 1 000 passos com coeficiente 0,2, em cinco execuções do programa.

Em cada execução, as três versões foram medidas nessa ordem, com a mesma condição inicial. O intervalo de `MPI_Wtime` abrange os passos da simulação e a barreira final; não inclui alocação, inicialização da barra nem redução da soma final. A soma global foi impressa como verificação simples de consistência entre versões. Para comparar os tempos, usa-se a mediana das cinco execuções.

## 4 Resultados

| Execução | Send/Recv (s) | Isend/Irecv+Wait (s) | Isend/Irecv+Test (s) |
|----------|---------------|----------------------|----------------------|
| 1        | 0,506098      | 0,498725             | 0,680938             |
| 2        | 0,520909      | 0,505166             | 0,682729             |
| 3        | 0,486050      | 0,479143             | 0,677186             |
| 4        | 0,517211      | 0,490103             | 0,678906             |
| 5        | 0,519315      | 0,500133             | 0,677699             |
| Mediana  | 0,517211      | 0,498725             | 0,678906             |

**Tabela 1:** Tempos do job 2109344. A soma final impressa foi 524 272,535328 em todas as versões e execuções.



**Figura 3:** Mediana das cinco execuções para cada versão.

As medianas das versões bloqueante e não bloqueante seguida de espera diferem em aproximadamente 3,6%, com menor tempo para a versão com `MPI_Isend/MPI_Irecv` e `MPI_Wait`. A versão com `MPI_Test` teve mediana aproximadamente 31% maior que a bloqueante: a sobreposição implementada não trouxe ganho neste caso. Uma hipótese é que o custo dos testes periódicos e sua interação com o cálculo superem o tempo de comunicação oculto; este experimento não isolou esses custos. Cada borda contém apenas um `double` (8 bytes), portanto a comunicação entre nós é dominada por mensagens pequenas e latência.

As cinco execuções apresentaram valores próximos, sem uma execução discrepante como no teste anterior. A soma final igual é uma verificação global, não uma comparação célula a célula.

## Anexo A: Código completo

```
1 #include <mpi.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 #define CELLS 1048576
6 #define STEPS 1000
7 #define COEFFICIENT 0.2
8
9 static void update(double *next, const double *current, int i) {
10     next[i] = current[i] + COEFFICIENT *(current[i - 1] - 2.0 * current[i] +
11     current[i + 1]);
12 }
13
14 static void blocking(double *next, double *current, int count, int left, int right)
15 {
16     MPI_Send(&current[count], 1, MPI_DOUBLE, right, 0, MPI_COMM_WORLD);
17     MPI_Recv(&current[0], 1, MPI_DOUBLE, left, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE
18     );
19     MPI_Send(&current[1], 1, MPI_DOUBLE, left, 1, MPI_COMM_WORLD);
20     MPI_Recv(&current[count + 1], 1, MPI_DOUBLE, right, 1, MPI_COMM_WORLD,
21     MPI_STATUS_IGNORE);
22
23     for (int i = 1; i <= count; ++i) update(next, current, i);
24 }
25
26 static void nonblocking(double *next, double *current, int count, int left, int
27 right) {
28     MPI_Request requests[4];
29     MPI_Irecv(&current[0], 1, MPI_DOUBLE, left, 0, MPI_COMM_WORLD, &requests[0]);
30     MPI_Irecv(&current[count + 1], 1, MPI_DOUBLE, right, 1, MPI_COMM_WORLD, &
31     requests[1]);
32     MPI_Isend(&current[count], 1, MPI_DOUBLE, right, 0, MPI_COMM_WORLD, &requests
33     [2]);
34     MPI_Isend(&current[1], 1, MPI_DOUBLE, left, 1, MPI_COMM_WORLD, &requests[3]);
35     for (int i = 0; i < 4; ++i) MPI_Wait(&requests[i], MPI_STATUS_IGNORE);
36
37     for (int i = 1; i <= count; ++i) update(next, current, i);
38 }
39
40 static void overlap(double *next, double *current, int count, int left, int right)
41 {
42     MPI_Request requests[4];
43     MPI_Irecv(&current[0], 1, MPI_DOUBLE, left, 0, MPI_COMM_WORLD, &requests[0]);
44     MPI_Irecv(&current[count + 1], 1, MPI_DOUBLE, right, 1, MPI_COMM_WORLD, &
45     requests[1]);
46     MPI_Isend(&current[count], 1, MPI_DOUBLE, right, 0, MPI_COMM_WORLD, &requests
47     [2]);
48     MPI_Isend(&current[1], 1, MPI_DOUBLE, left, 1, MPI_COMM_WORLD, &requests[3]);
49
50     int received_left = 0, received_right = 0;
51     for (int i = 2; i < count; ++i) {
```

```

42     update(next, current, i);
43     if (i % 1024 == 0) {
44         if (!received_left)
45             MPI_Test(&requests[0], &received_left, MPI_STATUS_IGNORE);
46         if (!received_right)
47             MPI_Test(&requests[1], &received_right, MPI_STATUS_IGNORE);
48     }
49 }
50 if (!received_left) MPI_Wait(&requests[0], MPI_STATUS_IGNORE);
51 if (!received_right) MPI_Wait(&requests[1], MPI_STATUS_IGNORE);
52 update(next, current, 1);
53 update(next, current, count);
54 MPI_Wait(&requests[2], MPI_STATUS_IGNORE);
55 MPI_Wait(&requests[3], MPI_STATUS_IGNORE);
56 }
57
58 int main(int argc, char **argv) {
59     int rank, process_count;
60     MPI_Init(&argc, &argv);
61     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
62     MPI_Comm_size(MPI_COMM_WORLD, &process_count);
63     if (process_count < 2 || process_count > CELLS / 2) {
64         if (rank == 0) fprintf(stderr, "Use de 2 a %d processos.\n", CELLS / 2);
65         MPI_Finalize();
66         return EXIT_FAILURE;
67     }
68
69     int count = CELLS / process_count + (rank < CELLS % process_count);
70     int start = rank * (CELLS / process_count) + (rank < CELLS % process_count ?
71         rank : CELLS % process_count);
72     int left = rank == 0 ? MPI_PROC_NULL : rank - 1;
73     int right = rank == process_count - 1 ? MPI_PROC_NULL : rank + 1;
74     double *a = calloc((size_t)count + 2, sizeof(double));
75     double *b = calloc((size_t)count + 2, sizeof(double));
76     if (!a || !b) {
77         fprintf(stderr, "Falha de alocacao no processo %d.\n", rank);
78         MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
79     }
80
81     void (*versions[])(double *, double *, int, int, int) = {
82         blocking, nonblocking, overlap
83     };
84     const char *names[] = {"Send/Recv", "Isend/Irecv+Wait", "Isend/Irecv+Test"};
85     if (rank == 0) printf("Versao\tTempo(s)\tSoma final\n");
86     for (int version = 0; version < 3; ++version) {
87         for (int i = 1; i <= count; ++i)
88             a[i] = start + i - 1 < CELLS / 2 ? 1.0 : 0.0;
89         a[0] = a[count + 1] = 0.0;
90         b[0] = b[count + 1] = 0.0;
91         double *current = a, *next = b;
92
93         MPI_Barrier(MPI_COMM_WORLD);
94         double begin = MPI_Wtime();

```

```

94     for (int step = 0; step < STEPS; ++step) {
95         versions[version](next, current, count, left, right);
96         double *temporary = current;
97         current = next;
98         next = temporary;
99     }
100     MPI_Barrier(MPI_COMM_WORLD);
101     double elapsed = MPI_Wtime() - begin;
102
103     double local_sum = 0.0, total_sum = 0.0;
104     for (int i = 1; i <= count; ++i) local_sum += current[i];
105     MPI_Reduce(&local_sum, &total_sum, 1, MPI_DOUBLE, MPI_SUM, 0,
106               MPI_COMM_WORLD);
107     if (rank == 0) printf("%s\t%.6f\t%.6f\n", names[version], elapsed,
108                           total_sum);
109 }
110
111 free(a);
112 free(b);
113 MPI_Finalize();
114 return EXIT_SUCCESS;
115 }

```

## Anexo B: Script Slurm

```

1  #!/bin/bash
2  #SBATCH --job-name=tarefa15
3  #SBATCH --output=tarefa15-%j.out
4  #SBATCH --nodes=2
5  #SBATCH --ntasks=2
6  #SBATCH --ntasks-per-node=1
7  #SBATCH --cpus-per-task=1
8  #SBATCH --time=00:10:00
9  #SBATCH --partition=intel-512
10
11  set -e
12  mpicc -std=c11 -O3 -Wall -Wextra -o tarefa15 tarefa15.c
13  for run in 1 2 3 4 5; do
14      echo "Execucao $run"
15      mpirun -np "$SLURM_NTASKS" ./tarefa15
16  done

```